1 Lecture 1: Introduction and Basic Security Concepts

1.2 CIA Triad and Examples

Confidentiality: prevent unauthorized disclosure.
Integrity: prevent unauthorized modification.
Availability: service/data remains usable.
Exam mapping: DoS targets availability; tampering targets integrity; eavesdropping/data leak targets confidentiality.

1.3 Encryption and Decryption: Basic Notation

Encryption: $CT = E(K, PT)$.
Decryption: $PT = E^{-1}(K, CT)$ or $D(K, CT)$.
Key point: encryption alone gives confidentiality, not necessarily integrity or authenticity.

1.4 Kerckhoffs's Principle

Principle: security must rely on secrecy of the *key*, not secrecy of the algorithm.
Exam point: do not count "algorithm is hidden" as a proper cryptographic defense.

1.6 Attacker Capabilities

CTO: ciphertext-only; given CTs, may know PT properties/distribution.
KPA: known plaintext; given PT-CT pairs, but cannot choose PT.
CPA: chosen plaintext; can query encryption oracle.
CCA2: chosen ciphertext; can query decryption oracle.
Ordering: CCA2 is stronger than CPA, which is stronger than KPA, which is stronger than CTO.

2 Lecture 2: Classical and Modern Ciphers

2.1 Substitution Cipher

Idea: replace each symbol with another symbol according to a key/permutation.
Weakness: preserves frequency patterns; vulnerable to frequency analysis.

2.2 Permutation / Transposition Cipher

Idea: rearrange positions of plaintext characters/blocks.
Weakness: does not hide character frequencies well; structure may remain.

2.3 Exhaustive Search and Key Space

Brute force: try every possible key.
Key space: larger key space increases exhaustive-search cost.
Exam point: large key space alone is not enough if cipher leaks structure.

2.4 One-Time Pad (OTP)

Key: truly random, same length as plaintext, used once only, destroyed after use.
Encryption: $C = P \oplus K$.
Decryption: $P = C \oplus K$.
Security: perfect secrecy if OTP conditions hold.
Weakness: impractical key distribution/storage.

2.5 Stream Cipher and IV

Stream cipher: $C = P \oplus K_s$, $P = C \oplus K_s$.
Keystream reuse: if $U = X \oplus K$ and $V = Y \oplus K$, then $U \oplus V = X \oplus Y$, leaking relation between plaintexts.
With IV: $C = P \oplus PRG(k, IV)$; send (CT, IV); IV need not be secret.
Rule: same key $\neq$ same keystream. Use nonce/IV to avoid reuse.

2.6 Block Ciphers and Modes of Operation

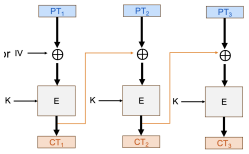
Block cipher: deterministic keyed permutation on fixed-size blocks.
Mode of operation: method for encrypting multi-block messages using a block cipher.

2.6.1 ECB

ECB: same plaintext block $\rightarrow$ same ciphertext block.
Weakness: leaks patterns; not semantically secure for repeated blocks.

2.6.2 CBC

$CT_j = E_K(PT_j \oplus CT_{j-1})$
$CT_0 = IV$
$PT_j = E_K^{-1}(CT_j) \oplus CT_{j-1}$



Encryption: sequential.
Decryption: parallelizable.
IV: random/unpredictable IV for semantic security.
Padding: required for non-block-aligned messages.
Malleability: changing CT_{j-1} flips bits in PT_j; changing IV flips bits in PT_1.

2.6.3 CTR

$X_1 = IV$, $X_j = X_1 + j - 1$.
$CT_j = E_K(X_j) \oplus PT_j$, $PT_j = E_K(X_j) \oplus CT_j$.
Padding: no padding.
Parallelism: parallelizable.
Nonce/counter reuse: reuses keystream.
CTR bug: if counter is not incremented, all blocks use same keystream. If one plaintext block is known, recover keystream, then recover all blocks.

2.7 Birthday Bound for IV/Hash Collisions

For an n-bit random IV/nonce/hash space, collision becomes noticeable around $q \approx 2^{n/2}$.
$\Pr[\text{collision}] \approx 1 - e^{-q(q-1)/(2N)}$, where $N = 2^n$.
Exam estimate: for probability around 0.4, use $q \approx 2^{n/2}$.

2.8 Meet-in-the-Middle Attack

Double encryption: $c = E_{k_2}(E_{k_1}(m))$. Naive exhaustive search: 2^{2k}.
MITM: compute/store $E_{k_1}(m)$ for all k_1; compute $D_{k_2}(c)$ for all k_2; match in middle.
Cost: ops $\approx 2^{k+1}$, storage $\approx 2^{k+1}$.
Takeaway: 2DES much weaker than 2^{112} intuition.

3 Lecture 3: Public-Key Cryptography and Integrity

3.1 Why Public-Key Cryptography

Problem solved: secure communication without pre-shared symmetric key.
Public key: shared publicly for encryption verification.
Private key: kept secret for decryption/signing.
Hybrid use: public-key crypto is expensive, so use it to protect small session keys.

3.2 Classroom RSA Key Generation

1. Choose two large primes p, q.
2. Compute $n = pq$.
3. Compute $\phi(n) = (p-1)(q-1)$.
4. Choose e such that $\gcd(e, \phi(n)) = 1$.
5. Find d such that $de \equiv 1 \pmod{\phi(n)}$.
6. Public key: $\langle n, e \rangle$. Private key: $\langle n, d \rangle$.

3.3 RSA Encryption and Decryption

Message: represented as integer $m < n$.
Encrypt: $c = m^e \bmod n$.
Decrypt: $m = c^d \bmod n$.
Correctness: $(m^e)^d = m^{ed} \equiv m \pmod n$.
Security intuition: from public $\langle n, e \rangle$, attacker needs d; deriving d requires $\phi(n)$; deriving $\phi(n)$ requires factoring n into p, q.

3.4 RSA Malleability and OAEP

Multiplicative malleability: if $c_1 = m_1^e \bmod n$ and $c_2 = m_2^e \bmod n$, then $c_1 c_2 \bmod n = (m_1 m_2)^e \bmod n$.
Exploit idea: attacker may alter ciphertext in a structured way so decrypted plaintext is predictably related to the original.
Integrity: malleability violates ciphertext integrity; encryption alone does not imply authenticity.
Determinism: raw RSA encrypts same message to same ciphertext.
OAEP: randomized padding reduces deterministic encryption and chosen-ciphertext style attacks.

3.5 Hybrid Encryption

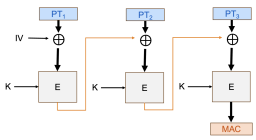
Step 1: generate random symmetric key K.
Step 2: encrypt data with AES using K.
Step 3: encrypt K using receiver's public key.
Derived point: hybrid encryption is used because RSA is slow and handles only small messages conveniently.

3.6 Hash Functions

Hash: unkeyed function $H(F) = h$ maps arbitrary-length file/message F to fixed-size digest h.
Hash alone: gives integrity checking only if trusted digest is available.
Preimage resistance: given h, hard to find m s.t. $H(m) = h$.
Second-preimage resistance: given m, hard to find $m' \neq m$ with same hash.
Collision resistance: hard to find any $m \neq m'$ with same hash.
Birthday attack: n-bit digest gives collisions around $2^{n/2}$, not 2^n.

3.7 MAC: Message Authentication Code

MAC: symmetric-key authentication tag; $t = mac_k(m)$.
Key: shared secret key.



Security: gives integrity + data-origin authenticity.
No non-repudiation: receiver also knows key and could have produced tag.
CBC-MAC caveat: safe only under correct conditions, e.g. fixed-length messages and no attacker-controlled IV.

3.8 HMAC

HMAC form: $\text{HMAC}_K(x) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel x))$.
Use: HMAC-SHA256/SHA512 for new applications.

3.9 Digital Signatures

Signing: sign with private key, verify with public key.
Security: gives public verifiability + non-repudiation.
Key asymmetry: anyone with public key can verify, but only private-key holder should be able to sign.

3.10 Hash-and-Sign

Reason: RSA is slow, so sign digest, not full file.
Digest: compute $h = H(X)$.
Sign: $s = RSA_encrypt(private\ key, H(X))$.
Verify: check $H(X) = RSA_dec(public\ key, s)$.
Exam distinction: MAC = shared-key authenticity; signature = public verifiability + non-repudiation.

4 Lecture 4: Password Authentication

4.1 Authentication Types and Credentials

Communicating entity: "who" is communicating.
Data origin: "where" data comes from.
Something you know: password.
Something you are: fingerprint / biometric.
Something you have: physical key / device.

4.2 Password System

Username: identity claim; not secret.
Password: credential used to authenticate user.
Risk: human-chosen passwords often have low entropy and are reused.

4.3 Secure Password File

Store passwords as: $(s,\ H(s \| p))$ with random salt s, never plaintext.
Stored salt: salt is stored with the password hash; it is not normally secret.
Precomputed tables: salt defeats direct reuse of precomputed hash tables.
Same password: salt prevents same password $\rightarrow$ same stored hash across users.
Offline cost: guesses must be recomputed per user/salt.
Salt limit: salt does not fix weak passwords; weak passwords remain vulnerable to offline guessing.

4.4 Password Attacks

Online dictionary: interacts with server; mitigate by rate limit, delay, lockout.
Offline dictionary: attacker has password file/hash; mitigate with salt and high entropy.
Password reset risk: security questions can be guessed from public information.

4.5 Entropy

Formula: if alphabet size is N and length is L, then $H \approx L \log_2 N$.
Example: $N = 94, L = 8 \Rightarrow H \approx 8 \log_2(94) \approx 52.4$ bits.
Online guidance: at least 29 bits; about 36–59 bits for higher security.
Offline guidance: lecture takes 128 bits as the target when offline attack is possible.

4.6 Multi-Step and Multi-Factor Authentication

Multi-step: more than one authentication step.
MFA: uses different factor categories, e.g. know + have or know + are.
Exam point: two passwords are two steps but not necessarily two factors.

4.7 Passkeys

Password: something you know; phishing risk.
Passkey: device-held key pair + PIN/biometric; private key stays on device; phishing-resistant.
Benefit: server does not store password; authentication is based on public-key challenge response.

5 Lecture 5: Public Key Infrastructure

5.1 Motivation: Public-Key Distribution

Problem: attacker may substitute their public key for victim's key.
Need: trusted binding between identity/name and public key.

5.2 Certificate Authority and Digital Certificate

Certificate: binds identity to public key, signed by CA: $\text{cert}_{C \rightarrow A} = \{A, K_A, \text{validity}, \ldots\}_{K_C^{-1}}$.
Issuance: CA verifies the claimed binding before issuing certificate.
Verification: client verifies CA signature using CA public key / chain of trust.
DV certificate: proves control of domain name only.
EV certificate: stricter identity validation; higher assurance but more cost/time.

5.3 Root CA and Chain of Trust

Root CA: preloaded in OS/browser.
Chain: browser verifies certificate chain up to trusted root CA.
Intermediate CA: verified through the chain, then signs end-entity certificates.

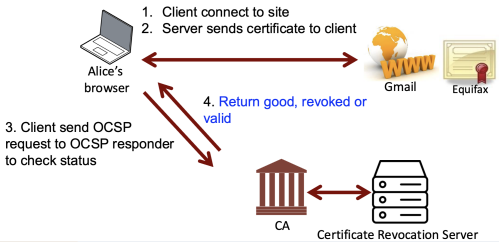
5.4 PKI Models

Single CA: easy to manage but single point of failure and poor scalability.
Multiple CAs: common in browsers; redundancy but more complexity.
Hierarchy: trusted root signs intermediate CAs, which sign domain/service certificates.

Weakest link: compromise of one trusted CA can undermine security.

5.5 Certificate Revocation

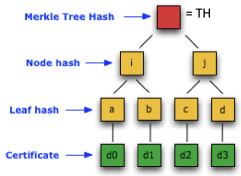
CRL / Delta CRL: Certificate Revocation List / only changes.
OCSP: Online Certificate Status Protocol.
OCSP stapling: server requests and sends OCSP response.
OCSP extension: client queries to get status.



Short-lived certificates: reduce revocation window.

5.6 Certificate Transparency

Cert log: public, append-only, cryptographically verifiable log that helps detect mis-issued / rogue certificates.
Merkle tree



src

SCT: Signed Certificate Timestamp; **not** proof of inclusion yet.
Promise: signed promise that log will include certificate within MMD.
Proof of inclusion: certificate hash + neighbor hashes up to root.
Exam distinction: SCT = promise; inclusion proof = Merkle path showing cert is already in log.
Neglected point: CT does not prevent a bad certificate from being issued; it makes bad issuance visible/auditable.

6 Lecture 6: Authenticated Key Exchange, TLS, and QUIC

6.1 Authentication Protocols

Goal: verify identity of communicating entity.
Replay problem: if credential is static and sniffable, attacker can resend it.
Freshness: use nonce/challenge to prevent replay.

6.2 Challenge–Response with MAC

1. Prover: "I am Alice"
2. Verifier sends fresh random nonce m
3. Prover sends $t = \mathrm{MAC}_k(m)$
4. Verifier checks tag

Why it works: attacker cannot forge MAC on unseen nonce.
Unilateral auth: only one side authenticated.
Neglected point: if challenge m is reused or predictable, replay may become possible.

6.3 Public-Key Authentication

1. Claim identity + certificate
2. Verifier sends random challenge r
3. Prover signs r using private key
4. Verify certificate + signature using public key

Benefit: no shared secret required.
Certificate role: binds claimed identity to public key.

6.4 Key Exchange Without Authentication

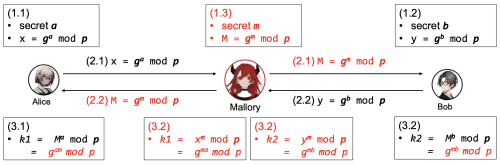
Problem: passive Eve only sniffs; active Mallory can modify messages and do MITM.
Key exchange alone: establishes a key but does not prove who contributed the values.

6.4.1 PKC-Based Key Transport

Alice: has long-term key pair $(k^e_{Alice}, k^d_{Alice})$ and certificate.
Identity: Alice sends "I am Alice", certificate.
Bob: chooses random session key k.
Transport: Bob computes $c = Encrypt(k^e_{Alice}, k)$ and sends c to Alice.
Alice decrypts: $k = Decrypt(k^d_{Alice}, c)$.
Authentication: only Alice should be able to decrypt c and obtain k.
Nature: key transport: Bob creates k and transports it to Alice.
Weakness: no PFS; later private-key compromise exposes old recorded transported keys.

6.4.2 Diffie–Hellman Key Exchange

Public parameters: prime p, generator g.
Alice: $x = g^a \bmod p$.
Bob: $y = g^b \bmod p$.
Shared key: $k = y^a \bmod p = x^b \bmod p = g^{ab} \bmod p$.
Security: based on hardness of discrete logarithm problem.
Exam numeric DH: compute powers mod p, reduce early and often.
Nature: key agreement: both sides contribute secrets to derive k.
Weakness: plain DH gives key exchange but not authentication; active MITM can replace x, y.



6.5 Authenticated Key Exchange and STS

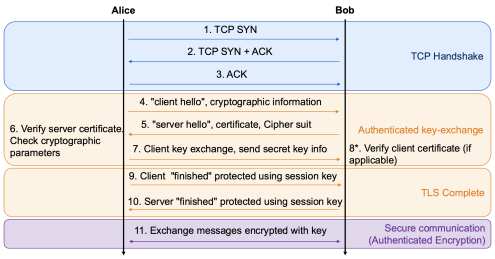
Goal: establish shared session key k and authenticate peer.
DHKE-based AKE: exchange DH values, then sign DH values/transcript using long-term private key.
Verification: verify signature using certified public key.
Session key: $k = g^{ab} \bmod p$.
Source of security: authentication from signature; secrecy from DH ephemeral secrets.
Long-term key: identity/authentication key, reused across sessions.
Session key: fresh key for one session; used for encrypted/MACed communication.
Neglected point: sign the DH transcript, not an unrelated message, otherwise MITM binding attacks may remain.

6.6 Perfect Forward Secrecy

Definition: compromise of long-term private key should *not* reveal past session keys.
DHKE PFS: fresh a, b per session; $k = g^{ab} \bmod p$ independent of long-term signing key; discard a, b, k.
PKC no PFS: recorded RSA-encrypted session keys become decryptable after
long-term private-key compromise.
Exam rule: ephemeral DH / DHE / ECDHE $\rightarrow$ PFS; PKC/RSA key transport $\not\rightarrow$ PFS.
Choice: prefer ECDHE/DHKE for modern secure channels; PKC key transport is legacy/simpler but unsuitable for recorded-traffic confidentiality.

6.7 TLS

TLS over TCP



TLS goal: secure channel over TCP with confidentiality, integrity, authentication.
Server proof: server proves possession of private key corresponding to certificate public key.
Client check: client verifies certificate public key belongs to intended domain.
Flow: TCP handshake; TLS handshake; certificate verification; key exchange; Finished messages; protected application data.

6.8 Authenticated Encryption: EtM, E&M, MtE

Let $C = E_{key1}(m_j)$.
EtM: $C \| mac_{key2}(C)$; preferred because ciphertext is authenticated before decryption.
E&M: $E_{key1}(m_j) \| mac_{key2}(m_j)$; MAC authenticates plaintext, not ciphertext; ciphertext malleability may remain.
MtE: $E_{key1}(m_j \| mac_{key2}(m_j))$; receiver decrypts before checking MAC; bad with padding-oracle style leakage.
Why EtM preferred: verify first, detect tampering early, reduce malleability/padding-oracle attack surface, use separate keys.
Neglected point: encryption alone only gives confidentiality. Authenticated encryption is needed to reject attacker-modified ciphertext.

6.10 QUIC

QUIC: Quick UDP Internet Connections.
Transport: runs over UDP.
Security: integrates TLS 1.3.
Latency: no separate TCP handshake; lower setup latency.
Comparison: TLS over TCP = TCP handshake + TLS handshake; QUIC integrates transport + TLS 1.3.
Neglected point: QUIC is not less secure because UDP; security is implemented at QUIC/TLS layer.

7 Lecture 7: Network Security

Core view: network attacker on path may sniff, spoof, modify, drop, redirect, or infer communication metadata.
Layer map: Application = HTTP/DNS; Transport = TCP/UDP ports; Network = IP; Data Link = MAC/ARP.
Addressing: IP is routable address; MAC is the local link-layer destination.
Port: open port means some process is listening at that transport-layer endpoint.
Visibility: TLS protects application data, but relevant nodes may still see IP/port/link metadata.

7.3 Network Attacker

Reliability $\neq$ security: TCP/IP being reliable does not stop MITM attacks.
MITM power: attacker at layer x can observe or alter the data unit visible at layer x.
High-yield examples: link layer: rogue AP / ARP poisoning; network layer: spoofing/routing abuse; application layer: DNS spoofing.

7.4 DNS Attacks

DNS basics: maps domain name $\rightarrow$ IP; client/resolver commonly queries over UDP.
QID weakness: matching the 16-bit query ID alone does not authenticate the response.
DNS spoofing: attacker races a forged answer; if it arrives first with matching QID, victim accepts attacker-controlled IP.
DNSSEC: adds authenticity and integrity via digital signatures.
DoH/DoT/DoQ: protect client-resolver transport; unlike DNSSEC, they do not authenticate DNS data itself.

7.5 Denial of Service Attack

Goal: break availability by overwhelming a server, service, or network.
DDoS: distributed attackers are stronger because one machine has limited sending rate.
Reflection: spoof victim IP so third parties send replies to the victim.
Amplification: small request triggers large response; common UDP amplifiers include DNS and NTP.
DNS target: taking down DNS can indirectly take down the web service.
Mitigation: redundancy, over-provisioning, rate limiting, filtering, and WAFs.

7.6 Poisoning ARP Table

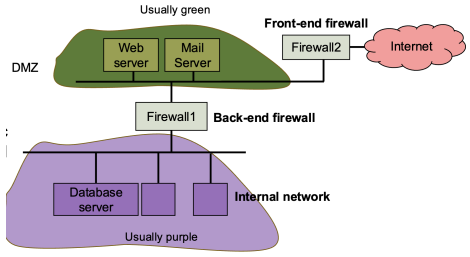
ARP basics: resolves local IP $\rightarrow$ MAC on the same subnet; hosts cache mappings in ARP tables.
Weakness: ARP has no authentication, so unsolicited spoofed replies may be accepted.
ARP poisoning: attacker binds victim IPs to attacker MAC and becomes a link-layer MITM.
Pattern: poisoning uses false ARP replies, not ARP requests.
Countermeasures: static ARP for critical hosts; Dynamic ARP Inspection on switches.
Exam distinction: DNS spoofing poisons name $\rightarrow$ IP; ARP poisoning poisons IP $\rightarrow$ MAC.

7.7 Securing Different Layers

Main idea: TLS/SSL, IPSEC, and WPA2 secure different layers, so they protect different information.
TLS/SSL: protects application payload above TCP; IP/port/link metadata can still remain visible.
IPSEC: protects network-layer traffic and is commonly used for VPN tunnelling.
WPA2: protects only the local Wi-Fi link between device and access point.
Comparison: TLS protects application content; IPSEC can hide more route-level traffic; WPA2 protects only the local hop.
Typical Implementation: TLS/SSL + WPA2 because IPSec is expensive to implement and difficult to deploy

7.8 Firewalls and IDS

Firewall: filters ingress/egress traffic; packet filtering inspects headers, DPI inspects payload too.
Demilitarize Zone: a sub-network exposing org's external service to Internet; Separates LAN from untrusted external networks
Defense in depth: a compromised DMZ host should still face a back-end firewall before reaching the LAN.



IDS: monitors packets/logs for malicious activity or policy violations.
Detection styles: signature-based = known patterns; anomaly/behavior-based = deviations from normal activity.

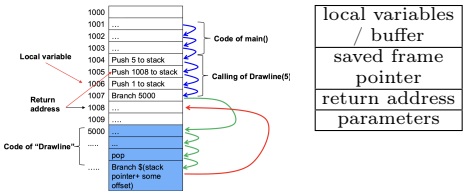
8 Lecture 8: Call Stack and Secure Programming

8.2 Control Flow Integrity

Execution integrity: only trusted and authorized code should execute.
Memory integrity: memory content should not be modified without authorization.
Control-Flow Integrity: program execution should remain within intended control-flow paths.
Interdependence: memory corruption can compromise control flow, e.g. by changing return address, function pointer, branch target, or injected code.
Attack mechanisms
Attack 1: replace code: overwrite existing normal code with malicious code; later branch executes attacker code.
Attack 2a: direct jump target: modify code/instruction target so direct branch jumps to attacker-chosen existing code/library.
Attack 2b: indirect jump target: modify memory location holding target address used by indirect branch/return; common via buffer overflow.
Return-to-libc idea: even if injected code cannot run, attacker may redirect control to existing library code with malicious effect.

8.3 Call Stack

Call stack: stack tracking active function calls; stack is memory data structure, not separate hardware.



Call flow: caller pushes parameters + return address; callee creates frame; return pops frame and branches to return address.
Attack target: local buffer overflow may overwrite saved frame pointer and return address, changing control flow.

- Memory Layout

Text section: program code/instructions.
Data section: global/static variables.
Heap: dynamic memory allocation.
Stack: function parameters, return addresses, local variables.
Exam point: stack smashing targets stack; heap overflow targets heap; both are memory corruption but may affect different control/data objects.

8.4 Secure Programming

Secure programming: write software that behaves correctly even under malicious input.
Main insecure practices: buffer overflow/stack smashing, integer overflow, unsafe functions, data-representation mismatch.

8.5 Buffer Overflow / Stack Smashing

Buffer overflow: writing beyond buffer boundary.
C/C++ issue: no automatic runtime bounds checking; efficient but error-prone.
Array example: if `int a[5]; int b;` are adjacent, writing `a[5]=3;` may overwrite `b`.
strcpy risk: `strcpy` copies whole source string until null terminator and ignores destination buffer size. Example danger:

```
char s1[10];
strcpy(s1, s2);
```

Overflow size: if `s2` has 11 chars plus `\0`, bytes beyond `s1[9]` overwrite adjacent memory.
Stack smashing: buffer overflow on stack; may overwrite local variables, saved frame pointer, or return address. Note: input writes from lower address to higher address
Shellcode attack: well-crafted input may place shellcode in memory and overwrite return address to jump to it.
Control-flow hijack: if return address is overwritten, function returns to attacker-chosen address.
Important nuance: buffer overflow is not automatically code execution; it becomes control-flow hijack only if overwritten data affects PC/return address/function pointer/branch target.

8.6 Integer Overflow

Fixed-size integers: arithmetic may wrap modulo word size.
Example: for 8-bit unsigned integer, $254 + 2 \equiv 0 \pmod{256}$.
Broken assumption: statements like $a < a + 1$ may fail after overflow.
Bounds-check bypass: integer overflow may make safety check pass, then buffer overflow occurs later.

8.7 Unsafe Functions: Format String

Unsafe printf: `printf(user_input);` treats attacker input as the format string, not ordinary data.
Safe printf: `printf("%s", user_input);` fixes the format string and treats input as data.
Root cause: `printf` is variadic: the format string decides how many arguments to read and how to interpret them. If arguments are missing, stack/register garbage may be treated as arguments.
Read leak: `%x`/`%d` print stack values; `%s` treats a stack value as a pointer and prints memory there.
Crash: repeated `%s` may dereference invalid addresses and segfault.
Write primitive: `%n` writes the number of printed characters to an address argument; dangerous if attacker controls the target address.
Impact: information leak, denial of service, or memory modification; worse if program has elevated privilege.

8.8 Data Representation: SSL Bug

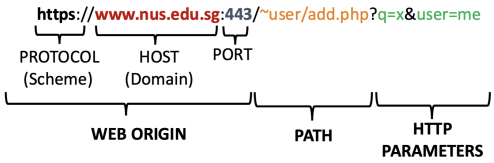
C string representation: string length not stored explicitly; first null byte `\0` marks end.
Representation mismatch: different components may interpret same byte sequence differently.
SSL-style bug: one component treats `canvas.nus.edu.sg\0.attacker.com` as ending at null; another validates or stores full byte string differently.
Risk: certificate/hostname validation may approve one name while display or later use treats it as another.

8.9 Countermeasures

Input validation/filtering: attack is triggered by carefully crafted input; validate user input immediately, reject unexpected format. Canonicalisation. Whitelist/Blacklist
Safe functions: avoid known-dangerous functions; use safer forms such as `printf("%s", user_input)` instead of `printf(user_input)`.
Bound checks: runtime checks compare array index with lower/upper bounds; halt if out of range; prevents buffer overflow but adds overhead.
Canaries: secret values inserted near control data; checked before return; if overflow changes canary, program halts. Mainly protects stack overflows.
Canary limitation: performance overhead; mainly detects consecutive overwrite before return address, not every memory corruption.
Type safety: static/dynamic type checks ensure operations receive values of expected type; reduces misuse of memory/data representation.
Testing: security testing uses rare/malicious inputs; white-box = source known, black-box = source unknown, grey-box = partial knowledge.
ASLR: randomizes memory layout of stack, heap, libraries etc.; makes target addresses harder to predict; especially useful against stack-smashing style attacks.
Non-executable stack: stack is marked non-executable; injected shellcode on stack cannot run.

9 Lecture 9: Web Security

9.1 Background



GET: fetches a resource identified by URL; parameters often appear in URL.
POST: sends data in request body after HTTP headers; commonly used for form submission.
HTML: web page structure; tags and attributes define elements, forms, inputs, links, etc.

9.2 Cookies and Attributes

Why cookies: HTTP is stateless; cookies allow server/browser to maintain session state.
Cookie session: server sets cookie; browser stores it; browser automatically sends matching cookie in later requests.
Token design: use random or MAC-protected tokens; predictable formats are insecure.
Cookie scoping: controls which domain, subdomain, and path receive the cookie; scope is ⟨domain, path⟩.
Secure: cookie only sent over HTTPS.
HttpOnly: cookie cannot be read by JavaScript, e.g. `document.cookie` fails.
SameSite: controls whether cookie is sent with cross-site requests.
Strict: cookie not sent with cross-site requests.
Lax: cookie may be sent on top-level navigation such as clicking a link.

```
Set-Cookie:  session=abc123;
path=/admin; domain=example.com;
secure; HttpOnly

Set-Cookie:  session=0F8tgdOhi9ynR1M9wa3ODa;
SameSite=Strict
```

9.3 Same-Origin Policy

Origin: scheme + hostname + port. Same origin iff all three match exactly.
Same-site vs same-origin: site is less specific (only accounts for scheme and last part of domain name); SameSite is for cookie sending; Same-Origin is for browser read/access control (general secuiry).
SOP purpose: prevents scripts from one origin reading data/resources from another origin.
SOP limitation: SOP restricts cross-origin reading, but does not stop cross-origin requests from being sent.Difference in SOP definition adopted by different browsers. Can be overly restrictive

9.4 Web Attacker Threat Model

Malicious web server: attacker is another end system; victim may be lured to attacker-controlled site.
Malicious client: attacker may interact with vulnerable server as a malicious user/client.
MITM attacker: attacker is in IP/lower layer, e.g. cafe Wi-Fi owner, DNS spoofing, or ARP poisoning; may impersonate server or inject traffic.

9.5 CSRF: Cross Site Request Forgery

Idea: attacker tricks authenticated victim browser into sending unwanted request to target site.
Why it works: authentication cookies are automatically included with requests to that domain, regardless of request source.
Attack requirement: victim logged in; authentication cookie present; victim visits/clicks attacker-controlled content.
Typical attack: hidden or auto-submitted form sends GET/POST to target site.
CSRF token: server sets unique token per session; every transaction includes token in hidden field; attacker cannot guess token, so forged form is rejected.
SameSite defense: SameSite=Strict stops cookie attachment on cross-site requests.
Exam clue: if CSRF fails, likely token missing/incorrect or SameSite prevented cookie attachment.

9.6 XSS

Reflected XSS: malicious input is immediately reflected in server response.
Persistent XSS: malicious script is stored on server and later served to victims.
Why dangerous: script executes in victim browser under target site's origin.
SOP and XSS: SOP does not stop malicious script already running inside the trusted origin.
HttpOnly caveat: HttpOnly blocks JavaScript from reading cookies, but malicious script may still send same-origin requests as the victim.
Defense: output encoding/escaping, input validation/sanitization, HttpOnly for cookies.

9.7 SQL Injection

Unsafe pattern:

```
"SELECT * FROM User WHERE"
"name = '" + userinput1 + "'"
"AND password = '" + userinput2 + "'"
```

Attack example:
`userinput1 = bob' OR 1=1; -`
Effect: `1=1` always true; `-` comments out password check; attacker may read unintended records.
Impact: confidentiality, integrity, or privilege escalation through a vulnerable app.
Mental model: code and data are mixed in one SQL string.
Defense: parameterized queries separate SQL command from parameters; parser treats parameters as data, never executable script.

10 Lecture 10: Access Control

10.1 Access Control Paradigm

Order: authenticate first, then authorize requested operation.
Subject / principal: human user or process acting on behalf of user.

Object: file, table, program, memory object, hardware device, etc.
Access mode: read, write, modify, delete, execute, create, copy, import/export, etc.
Access control function: checks authorization database/policy to decide whether authenticated subject may perform operation on object.

10.2 Access Control Implementation

Reference monitor: gatekeeper mediating subject requests to objects; grants/denies according to security policy.
Least privilege: give minimum permissions necessary for task.
Compartmentalization: separate sensitive resources/services into segments so compromise is contained.
Swiss Cheese / defense in depth: multiple imperfect layers; attack succeeds if holes align.
Segregation of duties: avoid single person having complete control over critical process/asset; reduces insider/single-point-of-failure risk.
DAC: object owner decides who can access object and what operations they may perform.
MAC: centralized system-wide policy; access based on subject clearance and object labels; not owner discretion.

10.3 Discretionary Access Control

Object

Principals	my.c	mysh.sh	sudo	a.txt
root	{r,w}	{r,x}	{r,s,o}	{r,w}
Alice	{}	{r,x,o}	{r,s}	{r,w,o}
Bob	{r,w,o}	{}	{r,s}	{}

ACL: matrix decomposed by columns; each object stores list of subjects and their rights.

my.c	→ (root, {r,w}) → (Bob, {r,w,o})
mysh.sh	→ (root, {r,x}) → (Alice, {r,x,o})
sudo	→ (root, {r,s,o}) → (Alice, {r,s}) → (Bob, {r,s})
a.txt	→ (root, {r,w}) → (Alice, {r,w,o})

Capability list: matrix decomposed by rows; each subject stores list of objects and rights.

root	→ (my.c, {r,w}) → (mysh.sh, {r,x}) → (sudo, {r,s,o}) → (a.txt, {r,w})
Alice	→ (mysh.sh, {r,x,o})→ (sudo, {r,s}) → (a.txt, {r,w,o})
Bob	→ (my.c, {r,w,o}) → (sudo, {r,s})

ACL tradeoff: easy to see who can access one object; harder to list all objects a subject can access.
Capability tradeoff: easy to see what one subject can access; harder to list all subjects who can access an object.

10.4 Mandatory Access Control

Bell–LaPadula: confidentiality model; no read up; no write down; read down/write up allowed.
Biba: integrity model; no read down; no write up; read up/write down allowed.
Biba goal: prevents lower-integrity subjects/data from contaminating higher-integrity data.
Exam hook: BLP = confidentiality; Biba = integrity.
Protection rings: lower ring number means higher privilege.
Ring 0: kernel / full access to hardware and system resources.
Ring 3: least privileged user applications.
Unix view: essentially two rings: superuser/kernel mode and user mode.

10.5 Intermediate Control

Intermediate control: balances coarse-grained controls and very fine-grained per-user permissions.
Group: subjects/objects grouped; rights defined on group to simplify matrix.
Role Based Access Control: permissions assigned to roles; users inherit permissions through roles.
Example: Canvas roles: Lecturer, TA, Student.
Design principle: role permissions should follow least privilege.

10.6 Controlled Invocation and Privilege Escalation

Bridge: predefined high-privilege executable mediates user access to sensitive file/resource.
Legitimate purpose: allows restricted interaction with data user cannot directly access.
Risk: if bridge has bug, attacker can craft input to make bridge perform illegal operation with elevated privilege.
SQL example: SQL server process anything from web server. If web server has no restriction on user's query, SQL injection may happen in privilege escalation case

10.7 Controlled Invocation in UNIX

RUID: real UID; inherited from user who invokes process; tracks who started process.
EUID: effective UID; used by reference monitor for access checks.
SUID disabled: EUID = RUID = invoking user.
SUID enabled: EUID becomes file owner's UID, e.g. root-owned executable runs with root EUID.

```
-rwxrw-r- 1 alice staff 23 Mar 13 00:27 temp
[type][owner/group/other][links][owner]
[group][size][time][name]
```

Bit position: SUID only applies to the owner execute bit: `-rwxrw-r- -> -rwsrw-r-`.
Use case: root-owned bridge such as `editprofile` can access protected file on behalf of normal user.
Risk: SUID program must be correct; unsafe input handling creates privilege escalation.

10.8 TOCTTOU Vulnerability

TOCTTOU: race condition between time permission/resource is checked and time it is used.
Unsafe pattern: program checks permission on filename, then later opens same filename.
Attack: attacker swaps file/symlink between `access()` and `open()`, so checked file differs from used file.
Defense 1: avoid separate check/use system calls on same filename; use `open()` once.
Defense 2: use `O_NOFOLLOW`; if path is symbolic link, `open()` fails.
Defense 3: set effective UID to appropriate target user before file access, so OS performs permission check directly.

11 Cross-Lecture Exam Synthesis

11.1 Confidentiality vs Integrity vs Authenticity

Encryption: confidentiality only unless authenticated.
MAC: integrity + authenticity under shared key, no non-repudiation.
Signature: integrity + authenticity + public verifiability + non-repudiation.
AEAD: confidentiality + integrity + authenticity.

11.2 Common Confusions

SOP: blocks reading cross-origin, not sending.
CSRF: uses browser auto-sent cookies.
XSS: script runs inside trusted origin.
Salt: stored; stops hash reuse/precomputation, not weak passwords.
SCT: promise to include, not proof of inclusion.
DNSSEC vs DoH/DoT: DNSSEC authenticates DNS data; DoH/DoT protect client-resolver channel.
ARP vs DNS poisoning: ARP poisons IP→MAC; DNS poisons name→IP.

11.3 High-Yield Formula Sheet

Entropy: $H \approx L \log_2 N$.
DH: $x = g^a \bmod p$, $y = g^b \bmod p$, $k = y^a = x^b = g^{ab} \bmod p$.
RSA keygen: $n = pq$, $\phi(n) = (p-1)(q-1)$, $\gcd(e, \phi(n)) = 1$, $de \equiv 1 \pmod{\phi(n)}$.
RSA enc/dec: $c = m^e \bmod n$, $m = c^d \bmod n$.
CBC: $CT_j = E_K(PT_j \oplus CT_{j-1})$, $PT_j = E_K^{-1}(CT_j) \oplus CT_{j-1}$.
CTR: $X_1 = IV$, $X_j = X_1 + j - 1$, $CT_j = E_K(X_j) \oplus PT_j$.
Birthday threshold: $q \approx 2^{n/2}$.
MITM on double encryption: ops $\approx 2^{k+1}$, storage $\approx 2^{k+1}$.
CBC first-block malleability: changing IV flips bits in PT_1 because $PT_1 = E_K^{-1}(CT_1) \oplus IV$.

11.4 If You See This, Think That

Clue	Think
same stream / CTR nonce	keystream reuse; XOR leakage
known CTR PT block	recover keystream; decrypt others
CBC + valid-padding oracle	padding oracle
RSA CT altered	malleability; no CT integrity
same raw RSA msg twice	deterministic; need OAEP/randomization
TLS domain-key proof	cert binding + private-key possession
SCT shown	promise, not inclusion proof
ephemeral DH	PFS
RSA key transport	no PFS
ARP spoofed reply	poison IP→MAC; MITM
browser auto-sends cookie	CSRF
stored script	persistent XSS
SQL string concat	SQL injection
user input as format	format-string vuln
`strncpy` long source	maybe no NUL terminator
DMZ	web/mail in DMZ; DB internal
unusual traffic	anomaly/behavior IDS
private docs leakage	Bell–LaPadula
poison trusted data	Biba
SUID	EUID becomes file owner
check then open	TOCTTOU

End · GYN2026